

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 8

Date: 28-08-2026

Implementation of Code Generation

Aim:

To implement a code generator that translates three address code containing assignments, arithmetic operators, relational operators, conditional constructs and loops into 8086 style assembly code.

Tools Used:

- GCC

Code:

codegen.c

```

/*
 * Code generator : Three Address Code -> 8086 style assembly code
 *
 * Supports : assignment, arithmetic operators (+ - * / %), unary minus,
 *            relational operators (< <= > >= == !=), conditional statements
 *            (if a relop b goto L) and iterative constructs (labels + goto).
 *
 * Method : 1. read the TAC and turn it into machine level instructions
 *           (constants that must sit in a register get their own MOV)
 *           2. live variable analysis (backward data flow, repeated until it
 *           stops changing - this is what makes loops work)
 *           3. two names may share a register only if they are never live at
 *           the same time; a name that finds no free register is spilled
 *           4. print the assembly code
 *
 * Input : x = y      x = y op z      x = -y      if a relop b goto L
 *         goto L      L: (a label may also precede a statement)
 * Usage : ./codegen < input.tac      or      ./codegen input.tac
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <stdarg.h>

#ifdef NREG
#define NREG 8 /* registers R0..R7 */
#endif
#define NALLOC (NREG - 2) /* R0..R5 are allocated, R6 and R7 are */
/* scratch registers used for spilled names */

#define NAME 32
#define MAXLAB 4
#define MAXSRC 200
#define MAXNODE 500
#define MAXVAR 300

/* ----- input TAC */

enum { S_COPY, S_BIN, S_NEG, S_IF, S_GOTO, S_NOP };

typedef struct {
    int kind;
    char dest[NAME], a[NAME], b[NAME]; /* x = a op b */
    char op[3]; /* arithmetic operator, or relational */

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char target[NAME];          /* jump target          */
char lab[MAXLAB][NAME];    /* labels attached to this statement */
int nlab;
} Src;

static Src src[MAXSRC];
static int nsrc = 0;
static char pend[MAXLAB][NAME]; /* labels waiting for their statement */
static int npend = 0;

static void die(const char *msg, int lineno, const char *text)
{
    printf("Error (line %d): %s : %s\n", lineno, msg, text);
    exit(1);
}

static char *skip(char *p)
{
    while (isspace((unsigned char)*p)) p++;
    return p;
}

/* read an identifier or a number (a '-' directly before digits is part of it) */
static char *word(char *p, char *out, int minus)
{
    int k = 0;
    p = skip(p);
    if (minus && *p == '-' && isdigit((unsigned char)p[1])) out[k++] = *p++;
    if (isdigit((unsigned char)*p))
        while (isdigit((unsigned char)*p) && k < NAME - 1) out[k++] = *p++;
    else if (isalpha((unsigned char)*p) || *p == '_')
        while ((isalnum((unsigned char)*p) || *p == '_') && k < NAME - 1) out[k++] = *p++;
    out[k] = '\0';
    return k ? p : NULL;
}

static int isnum(const char *s)
{
    return isdigit((unsigned char)s[0]) || (s[0] == '-' && isdigit((unsigned char)s[1]));
}

static int keyword(char *p, const char *kw)
{
    size_t n = strlen(kw);
    return !strncmp(p, kw, n) && !isalnum((unsigned char)p[n]) && p[n] != '_';
}

static void parse_line(char *p, int lineno)
{
    Src *s = &src[nsrc];
    char *q, w[NAME];
    const char *whole = p;

    memset(s, 0, sizeof *s);
    for (;;) {
        q = word(p, w, 0);
        if (q && !isdigit((unsigned char)w[0]) && *skip(q) == ':' && !keyword(w, "if")) {
            if (npend >= MAXLAB) die("too many labels", lineno, whole);
            strcpy(pend[npend++], w);
            p = skip(q) + 1;
        } else break;
    }
    p = skip(p);
    if (!*p) return;
    if (keyword(p, "goto")) {
        s->kind = S_GOTO;
    }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!(q = word(p + 4, s->target, 0)) || *skip(q)) die("bad goto", lineno, whole);
} else if (keyword(p, "if")) {
    s->kind = S_IF;
    if (!(p = word(p + 2, s->a, 1))) die("bad if", lineno, whole);
    p = skip(p);
    if (!strcmp(p, "<=", 2) || !strcmp(p, ">=", 2) ||
        !strcmp(p, "==", 2) || !strcmp(p, "!=", 2)) {
        strcpy(s->op, p, 2); p += 2;
    } else if (*p == '<' || *p == '>') {
        s->op[0] = *p++;
    } else die("relational operator expected", lineno, whole);
    if (!(p = word(p, s->b, 1))) die("bad if", lineno, whole);
    p = skip(p);
    if (!keyword(p, "goto")) die("goto expected", lineno, whole);
    if (!(q = word(p + 4, s->target, 0)) || *skip(q)) die("bad goto", lineno, whole);
} else {
    if (!(p = word(p, s->dest, 0)) || !isnum(s->dest)) die("bad statement", lineno, whole);
    p = skip(p);
    if (*p != '=' || p[1] == '=') die("'=' expected", lineno, whole);
    p = skip(p + 1);
    if (*p == '-' && !isdigit((unsigned char)p[1])) { /* x = -y */
        s->kind = S_NEG;
        if (!(p = word(p + 1, s->a, 0))) die("bad operand", lineno, whole);
    } else {
        if (!(p = word(p, s->a, 1))) die("bad operand", lineno, whole);
        p = skip(p);
        if (!*p) s->kind = S_COPY;
        else if (strchr("+-*/%", *p)) {
            s->kind = S_BIN;
            s->op[0] = *p++;
            if (!(p = word(p, s->b, 1))) die("bad operand", lineno, whole);
        } else die("operator expected", lineno, whole);
    }
    if (*skip(p)) die("unexpected text", lineno, whole);
}
memcpy(s->lab, pend, sizeof pend);
s->nlab = npend;
npend = 0;
nsrcc++;
}

/* ----- machine level code */

enum { N_MOVI, N_COPY, N_BIN, N_NEG, N_CMPJ, N_JMP, N_NOP };

typedef struct {
    int      kind;
    int      d, a, b;          /* variable numbers, -1 if unused */
    long long imm;            /* N_MOVI : constant loaded */
    char      op[3];           /* arithmetic or relational operator */
    char      tlabel[NAME];    /* N_CMPJ / N_JMP : target label */
    int      target;           /* ... and the node that carries it */
    char      lab[MAXLAB][NAME]; /* labels on this node */
    int      nlab;
} Node;

static Node node[MAXNODE];
static int  nn = 0;

static char names[MAXVAR][NAME]; /* variable table */
static int  isconst[MAXVAR];     /* 1 : register holding a constant */
static long long cval[MAXVAR];
static int  nvar = 0;

static int var(const char *name)
{
    int v;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    for (v = 0; v < nvar; v++)
        if (!isconst[v] && !strcmp(names[v], name)) return v;
    if (nvar >= MAXVAR) { printf("Error: too many variables\n"); exit(1); }
    strcpy(names[nvar], name);
    isconst[nvar] = 0;
    return nvar++;
}

static int newconst(long long value)
{
    if (nvar >= MAXVAR) { printf("Error: too many variables\n"); exit(1); }
    sprintf(names[nvar], "%lld", value);
    isconst[nvar] = 1;
    cval[nvar] = value;
    return nvar++;
}

static Node *add(int kind, int d, int a, int b)
{
    Node *n;
    if (nn >= MAXNODE) { printf("Error: program too long\n"); exit(1); }
    n = &node[nn++];
    memset(n, 0, sizeof *n);
    n->kind = kind; n->d = d; n->a = a; n->b = b;
    return n;
}

/* Operand of a statement as a variable number; a constant is first loaded into
   a register of its own (this MOV is emitted before the statement itself). */
static int operand(const char *s)
{
    int v;
    if (!isnum(s)) return var(s);
    v = newconst(atoll(s));
    add(N_MOVI, v, -1, -1)->imm = cval[v];
    return v;
}

static void attach(int at, const Src *s)
{
    memcpy(node[at].lab, s->lab, sizeof s->lab);
    node[at].nlab = s->nlab;
}

/* May the constants of the conditional jump 'i' be loaded before its label?
   Yes if every jump to that label comes from below (a loop back edge), because
   then the label is only reached again after the constant was loaded. */
static int hoistable(int i)
{
    int j, k;
    if (src[i].kind != S_IF || src[i].nlab == 0) return 0;
    for (j = 0; j < i; j++) {
        if (src[j].kind != S_IF && src[j].kind != S_GOTO) continue;
        for (k = 0; k < src[i].nlab; k++)
            if (!strcmp(src[j].target, src[i].lab[k])) return 0;
    }
    return 1;
}

static void build(int hoist)
{
    int i, first, a, b, d, at;
    nn = 0; nvar = 0;
    for (i = 0; i < nsrc; i++) {
        Src *s = &src[i];
        first = nn;
        switch (s->kind) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    case S_COPY:
        d = var(s->dest);
        if (isnum(s->a)) add(N_MOVI, d, -1, -1->imm = atoll(s->a);
        else add(N_COPY, d, var(s->a), -1);
        at = first;
        break;
    case S_NEG:
        d = var(s->dest);
        if (isnum(s->a)) add(N_MOVI, d, -1, -1->imm = -atoll(s->a);
        else add(N_NEG, d, var(s->a), -1);
        at = first;
        break;
    case S_BIN:
        a = operand(s->a); b = operand(s->b); d = var(s->dest);
        strcpy(add(N_BIN, d, a, b)->op, s->op);
        at = first;
        break;
    case S_IF:
        a = operand(s->a); b = operand(s->b);
        strcpy(add(N_CMPJ, -1, a, b)->op, s->op);
        strcpy(node[nn - 1].tlabel, s->target);
        at = (hoist && hoistable(i)) ? nn - 1 : first;
        break;
    case S_GOTO:
        strcpy(add(N_JMP, -1, -1, -1->tlabel, s->target);
        at = first;
        break;
    default:
        add(N_NOP, -1, -1, -1);
        at = first;
    }
    attach(at, s);
}
/* resolve the jump targets */
for (i = 0; i < nn; i++) {
    int k, m, found = 0;
    if (node[i].kind != N_CMPJ && node[i].kind != N_JMP) continue;
    for (k = 0; k < nn && !found; k++)
        for (m = 0; m < node[k].nlab; m++)
            if (!strcmp(node[k].lab[m], node[i].tlabel)) {
                node[i].target = k; found = 1; break;
            }
    if (!found) { printf("Error: label %s is not defined\n", node[i].tlabel); exit(1); }
}

/* ----- live variable analysis */

static char in[MAXNODE][MAXVAR], out[MAXNODE][MAXVAR]; /* live before / after a node */
static char exitLive[MAXVAR];

static int is_temp(const char *s) /* t1, t2 ... */
{
    int k;
    if (s[0] != 't' || !s[1]) return 0;
    for (k = 1; s[k]; k++) if (!isdigit((unsigned char)s[k])) return 0;
    return 1;
}

static void liveness(void)
{
    int changed = 1, k, v;
    memset(in, 0, sizeof in); memset(out, 0, sizeof out); memset(exitLive, 0, sizeof exitLive);
    for (v = 0; v < nvar; v++) /* results of the program stay alive */
        if (!isconst[v] && !is_temp(names[v])) exitLive[v] = 1;

    while (changed) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

changed = 0;
for (k = nn - 1; k >= 0; k--) {
    Node *n = &node[k];
    char o[MAXVAR], i2[MAXVAR];
    memset(o, 0, sizeof o);
    if (n->kind != N_JMP) { /* fall through to the next node */
        if (k + 1 < nn) for (v = 0; v < nvar; v++) o[v] |= in[k + 1][v];
        else for (v = 0; v < nvar; v++) o[v] |= exitLive[v];
    }
    if (n->kind == N_CMPJ || n->kind == N_JMP) /* jump to the target */
        for (v = 0; v < nvar; v++) o[v] |= in[n->target][v];
    memcpy(i2, o, sizeof i2);
    if (n->d >= 0) i2[n->d] = 0; /* in = use + (out - def) */
    if (n->a >= 0) i2[n->a] = 1;
    if (n->b >= 0) i2[n->b] = 1;
    if (memcmp(o, out[k], sizeof o) || memcmp(i2, in[k], sizeof i2)) {
        memcpy(out[k], o, sizeof o);
        memcpy(in[k], i2, sizeof i2);
        changed = 1;
    }
}
}

/* ----- register allocation */

static char interf[MAXVAR][MAXVAR];
static int reg[MAXVAR]; /* register number, -1 = not allocated */
static int spilled[MAXVAR]; /* 1 = lives in memory */

static void interfere(int x, int y)
{
    if (x != y) interf[x][y] = interf[y][x] = 1;
}

static void allocate(void)
{
    int order[MAXVAR], seen[MAXVAR], cnt = 0, k, v, w, r, c;
    memset(interf, 0, sizeof interf); memset(seen, 0, sizeof seen);
    for (v = 0; v < nvar; v++) { reg[v] = -1; spilled[v] = 0; }

    /* a name that is defined interferes with everything alive right after it
       (except the source of a copy, which holds the same value) */
    for (k = 0; k < nn; k++) {
        Node *n = &node[k];
        if (n->d < 0) continue;
        for (w = 0; w < nvar; w++)
            if (out[k][w] && !(n->kind == N_COPY && w == n->a)) interfere(n->d, w);
    }
    /* d = a - b : the result register must not be the register of b, otherwise
       b would be overwritten before it is used (+ and * can simply be swapped) */
    for (k = 0; k < nn; k++)
        if (node[k].kind == N_BIN && node[k].op[0] != '+' && node[k].op[0] != '*')
            interfere(node[k].d, node[k].b);
    /* names that are alive on entry (used before being assigned) all coexist */
    for (v = 0; v < nvar; v++)
        for (w = v + 1; w < nvar; w++)
            if (in[0][v] && in[0][w]) interfere(v, w);

    /* colour the names in the order in which they first appear */
    for (k = 0; k < nn; k++) {
        int list[3] = { node[k].a, node[k].b, node[k].d };
        for (c = 0; c < 3; c++)
            if (list[c] >= 0 && !seen[list[c]]) { seen[list[c]] = 1; order[cnt++] = list[c]; }
    }
    for (k = 0; k < cnt; k++) {
        int used[NALLOC + 1], pick = -1;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    v = order[k];
    memset(used, 0, sizeof used);
    for (w = 0; w < nvar; w++)
        if (interf[v][w] && reg[w] >= 0) used[reg[w]] = 1;
    /* x = y, x = -y, x = y op z : x likes to reuse the register of y */
    for (c = 0; c < nn && pick < 0; c++) {
        int p = -1;
        if (node[c].kind != N_COPY && node[c].kind != N_NEG && node[c].kind != N_BIN)
            continue;
        if (node[c].d == v) p = node[c].a; else if (node[c].a == v) p = node[c].d;
        if (p >= 0 && reg[p] >= 0 && !used[reg[p]]) pick = reg[p];
    }
    for (r = 0; r < NALLOC && pick < 0; r++)
        if (!used[r]) pick = r;
    if (pick >= 0) reg[v] = pick; else spilled[v] = 1;
}
}

/* ----- code emission */

#define S1 (NREG - 2)          /* scratch registers          */
#define S2 (NREG - 1)

static char pendlab[MAXLAB][NAME];
static int npendlab = 0;

static void emit(const char *fmt, ...)
{
    va_list ap;
    int k;
    for (k = 0; k < npendlab - 1; k++) printf("%s:\n", pendlab[k]);
    if (npendlab) printf("%s: ", pendlab[npendlab - 1]);
    npendlab = 0;
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    printf("\n");
}

static const char *opnd(int v)          /* how an operand is written */
{
    static char buf[4][NAME + 4];
    static int turn = 0;
    char *b = buf[turn++ & 3];
    if (spilled[v]) {
        if (isconst[v]) sprintf(b, "%lld", cval[v]);
        else             sprintf(b, "[%s]", names[v]);
    } else sprintf(b, "R%d", reg[v]);
    return b;
}

static const char *mnemonic(const char *op)
{
    switch (op[0]) {
        case '+': return "ADD";
        case '-': return "SUB";
        case '*': return "MUL";
        case '/': return "DIV";
        default : return "MOD";
    }
}

static const char *jump(const char *rel)
{
    if (!strcmp(rel, "<")) return "JL";
    if (!strcmp(rel, "<=")) return "JLE";
    if (!strcmp(rel, ">")) return "JG";

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!strcmp(rel, ">=")) return "JGE";
    if (!strcmp(rel, "==")) return "JE";
    return "JNE";
}

static void generate(void)
{
    int k, m;
    for (k = 0; k < nn; k++) {
        Node *n = &node[k];
        npendlab = 0;
        for (m = 0; m < n->nlab; m++) strcpy(pendlab[npendlab++], n->lab[m]);

        switch (n->kind) {
            case N_MOVI:
                if (isconst[n->d] && spilled[n->d]) break; /* used as an immediate */
                if (spilled[n->d]) emit("MOV %s, #lld", opnd(n->d), n->imm);
                else emit("MOV R%d, #lld", reg[n->d], n->imm);
                break;
            case N_COPY:
                if (!spilled[n->d] && !spilled[n->a] && reg[n->d] == reg[n->a]) break;
                if (spilled[n->d] && spilled[n->a]) {
                    emit("MOV R%d, %s", S1, opnd(n->a));
                    emit("MOV %s, R%d", opnd(n->d), S1);
                } else emit("MOV %s, %s", opnd(n->d), opnd(n->a));
                break;
            case N_NEG: {
                int r = spilled[n->d] ? S1 : reg[n->d];
                if (spilled[n->a] || reg[n->a] != r) emit("MOV R%d, %s", r, opnd(n->a));
                emit("NEG R%d", r);
                if (spilled[n->d]) emit("MOV %s, R%d", opnd(n->d), r);
                break;
            }
            case N_BIN: {
                int r = spilled[n->d] ? S1 : reg[n->d];
                int a_here = !spilled[n->a] && reg[n->a] == r;
                int b_here = !spilled[n->b] && reg[n->b] == r;
                int comm = (n->op[0] == '+' || n->op[0] == '*');
                if (b_here && !a_here && comm) { /* r already holds b */
                    emit("%s R%d, %s", mnemonic(n->op), r, opnd(n->a));
                } else if (b_here && !a_here) { /* x = y - x : r holds b */
                    emit("MOV R%d, %s", S1, opnd(n->a));
                    emit("%s R%d, R%d", mnemonic(n->op), S1, r);
                    emit("MOV R%d, R%d", r, S1);
                } else {
                    if (!a_here) emit("MOV R%d, %s", r, opnd(n->a));
                    emit("%s R%d, %s", mnemonic(n->op), r, opnd(n->b));
                }
                if (spilled[n->d]) emit("MOV %s, R%d", opnd(n->d), r);
                break;
            }
            case N_CMPJ:
                if (spilled[n->a]) { /* CMP needs a register first */
                    emit("MOV R%d, %s", S1, opnd(n->a));
                    emit("CMP R%d, %s", S1, opnd(n->b));
                } else emit("CMP %s, %s", opnd(n->a), opnd(n->b));
                emit("%s %s", jump(n->op), n->tlabel);
                break;
            case N_JMP:
                emit("JMP %s", n->tlabel);
                break;
            default:
                break;
        }
    }
    for (m = 0; m < npendlab; m++) /* labels of nodes that produced no code */
        printf("%s:\n", pendlab[m]);
    npendlab = 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    }
}

/* ----- main */

static void print_tac(void)
{
    int i, m;
    for (i = 0; i < nsrc; i++) {
        Src *s = &src[i];
        for (m = 0; m < s->nlab; m++)
            printf(s->kind == S_NOP && m == s->nlab - 1 ? "%s:" : "%s: ", s->lab[m]);
        switch (s->kind) {
            case S_COPY: printf("%s = %s", s->dest, s->a); break;
            case S_NEG:  printf("%s = -%s", s->dest, s->a); break;
            case S_BIN:  printf("%s = %s %s %s", s->dest, s->a, s->op, s->b); break;
            case S_IF:   printf("if %s %s %s goto %s", s->a, s->op, s->b, s->target); break;
            case S_GOTO: printf("goto %s", s->target); break;
            default: break;
        }
        printf("\n");
    }
}

int main(int argc, char *argv[])
{
    FILE *fp = (argc > 1) ? fopen(argv[1], "r") : stdin;
    char text[256];
    int lineno = 0, v, hoist, k, any;

    if (!fp) { perror(argv[1]); return 1; }
    while (fgets(text, sizeof text, fp)) {
        char *s = text, *c;
        lineno++;
        if ((c = strstr(s, "//")) != NULL) *c = '\0';
        if ((c = strchr(s, '#')) != NULL) *c = '\0';
        s[strcspn(s, "\r\n")] = '\0';
        s = skip(s);
        if (!*s) continue;
        if (nsrc >= MAXSRC - 1) die("program too long", lineno, s);
        parse_line(s, lineno);
    }
    if (npend) { /* labels at the very end */
        memset(&src[nsrc], 0, sizeof(Src));
        src[nsrc].kind = S_NOP;
        memcpy(src[nsrc].lab, pend, sizeof pend);
        src[nsrc].nlab = npend;
        nsrc++;
    }

    /* load loop constants before the loop if that is safe, otherwise inside */
    for (hoist = 1; ; hoist = 0) {
        build(hoist);
        liveness();
        for (v = 0, any = 0; v < nvar; v++) if (isconst[v] && in[0][v]) any = 1;
        if (!hoist || !any) break;
    }
    allocate();

    printf("Three Address Code\n");
    print_tac();
    printf("\nAssembly Code\n");
    generate();

    printf("\nRegister Allocation\n");
    for (v = 0; v < nvar; v++) {
        if (isconst[v]) continue;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    printf("    %-6s -> ", names[v]);
    if (spilled[v]) printf("memory [%s]\n", names[v]);
    else             printf("R%d\n", reg[v]);
}
for (v = 0; v < nvar; v++) {
    if (!isconst[v]) continue;
    for (k = 0, any = 0; k < nn; k++) if (node[k].kind == N_MOVI && node[k].d == v) any = 1;
    if (any && !spilled[v]) printf("    %-6s -> R%d\n", names[v], reg[v]);
}
for (v = 0; v < nvar; v++)
    if (!isconst[v] && in[0][v])
        printf("Note: %s is used before it is assigned (treated as an input)\n", names[v]);
return 0;
}

```

Input (input.txt) - three address code of $x=0$; for($i=1$; $i \leq 10$; $i++$) $x=x+1$

```

x=0
i=1
L3: if i<=10 goto L1
goto L2
L1: t1=x+i
x=t1
t2=i+1
i=t2
goto L3
L2:

```

Commands Used

```

gcc codegen.c -o codegen
./codegen < input.txt

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

#1      -> R3
● kushaal@Kushaals-MacBook-Air Assignment 8 % gcc codegen.c -o codegen
● kushaal@Kushaals-MacBook-Air Assignment 8 % cat input.txt
x=0
i=1
L3: if i<=10 goto L1
goto L2
L1: t1=x+i
x=t1
t2=i+1
i=t2
goto L3
L2:
● kushaal@Kushaals-MacBook-Air Assignment 8 % ./codegen < input.txt
Three Address Code
x = 0
i = 1
L3: if i <= 10 goto L1
goto L2
L1: t1 = x + i
x = t1
t2 = i + 1
i = t2
goto L3
L2:

Assembly Code
MOV R0, #0
MOV R1, #1
MOV R2, #10
L3: CMP R1, R2
JLE L1
JMP L2
L1: ADD R0, R1
MOV R3, #1
ADD R1, R3
JMP L3
L2:

Register Allocation
x      -> R0
i      -> R1
t1     -> R0
t2     -> R1
#10    -> R2
#1     -> R3

```

Assembly code produced for the given three address code

x = t1 and i = t2 need no instruction, because t1 and t2 are placed in the registers of x and i, which are no longer needed. The constant 10 does not change inside the loop, so it is loaded into R2 once, before the label L3.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

#1 -> R3
• kushaal@Kushaals-MacBook-Air Assignment 8 % cat input2.txt
a = 8
b = 3
if a > b goto L1
c = a - b
goto L2
L1: c = a * b
L2: t1 = c % 5
d = -t1
• kushaal@Kushaals-MacBook-Air Assignment 8 % ./codegen < input2.txt
Three Address Code
a = 8
b = 3
if a > b goto L1
c = a - b
goto L2
L1: c = a * b
L2: t1 = c % 5
d = -t1

Assembly Code
MOV R0, #8
MOV R1, #3
CMP R0, R1
JG L1
MOV R2, R0
SUB R2, R1
JMP L2
L1: MOV R2, R0
MUL R2, R1
L2: MOV R3, #5
MOV R4, R2
MOD R4, R3
NEG R4

Register Allocation
a    -> R0
b    -> R1
c    -> R2
t1   -> R4
d    -> R4
#5   -> R3

```

Another test case: conditional statement with arithmetic operators

Learning Outcomes:

- Learned how to translate three address code into target instructions (MOV, ADD, SUB, MUL, DIV, CMP and the jumps JL, JLE, JG, JGE, JE, JNE).
- Understood live variable analysis for programs with loops: it is a backward data flow problem that is repeated until nothing changes.
- Learned register allocation: two names can share a register only if they are never live together, and a name with no free register is kept in memory.
- Learned to translate relational operators and loops into a compare followed by a conditional jump, and to load loop-invariant constants before the loop only when that is safe.